Google sign-in and automatic organization sign-up

Both CloudGuard and GRC now provide `/login` and `/signup`. Users can continue with Google or register by email. Email registration continues to the Keycloak registration form, where passwords and email verification are handled. Google is brokered through Keycloak, so APIs continue accepting the same issuer, audience, and tenant claims.

After email verification, the first login automatically creates a separate organization and an administrator account for its owner. Existing members keep their organization and roles. Tenant IDs are generated by the server; users cannot select a tenant through the registration form. Concurrent requests and interrupted setup reuse the same database reservation.

Configure Google and Keycloak

1. In Google Cloud, configure the OAuth consent screen and create an OAuth client of type **Web application**. Add this authorized redirect URI, using the public Keycloak URL, including `/auth` if your installation uses it:

   ```
   https://auth.example.com/realms/gigachad-grc/broker/google/endpoint
   ```

2. Configure Keycloak SMTP so new email accounts can complete email verification. Keep Google consent-screen testing restrictions in mind when selecting test accounts.

3. Set the following environment variables in your local secure environment, then run the configuration script against the intended realm. Do not put secrets in frontend variables or commit them:

   ```
   KEYCLOAK_URL=https://auth.example.com
   KEYCLOAK_REALM=gigachad-grc
   KEYCLOAK_ADMIN_TOKEN=<short-lived Keycloak administration access token>
   GOOGLE_OAUTH_CLIENT_ID=<Google Web application client ID>
   GOOGLE_OAUTH_CLIENT_SECRET=<Google client secret>
   ```

   ```
   python3 grc-platform/auth/configure-google-signup.py
   ```

   The script first protects `organization_id` as an administrator-editable attribute, then creates/updates the Google provider under alias `google` and enables verified-email registration. It preserves unrelated user-profile attributes and does not print or write secrets. Realm JSON imports remain registration-disabled until this setup is applied. Do not enable registration without protecting the organization attribute.

Configure organization provisioning

The GRC controls service owns organization creation, using its existing PostgreSQL schema and a server-side Keycloak administration client. No new database migration is required for this feature.

Set on the controls service:

```
KEYCLOAK_ADMIN_CLIENT_ID=grc-services
KEYCLOAK_ADMIN_CLIENT_SECRET=<the matching confidential-client secret>
KEYCLOAK_ONBOARDING_AUDIENCES=grc-api,cloudguard-api
```

Use a confidential service-account client in the same realm. Grant its service account the Keycloak `realm-management` roles needed to read/update users and assign application realm roles (`manage-users` and `view-realm` with the default administration permissions). The application realm role `admin` must exist. If fine-grained administration permissions are enabled, explicitly allow the corresponding user and role-mapping operations. These are server credentials, distinct from Google's client secret and the public frontend clients.

Keep the existing `KEYCLOAK_URL`, `KEYCLOAK_REALM`, `KEYCLOAK_ISSUER`, `KEYCLOAK_JWKS_URL`, and API audience settings. The onboarding endpoint verifies signed RS256 tokens, issuer, audience, expiry, subject, and verified email before any database change. Ordinary API authentication still requires the organization claim.

For the CloudGuard API, configure a trusted URL reachable from its backend:

```
GRC_ONBOARDING_URL=https://grc.example.com/api/auth/onboard
```

On a shared private Compose network this may be `http://controls:3001/api/auth/onboard`. CloudGuard forwards the verified bearer token to that configured service; it never uses a caller-supplied destination. Use HTTPS when crossing hosts. Both applications must use the same Keycloak realm. Google access tokens are not accepted as application API tokens.

The public Keycloak clients must allow their actual application origins and callbacks:

Client	Callback
`cloudguard-frontend`	`https://cloudguard.example.com/login`
`grc-frontend`	`https://grc.example.com/`

Allow the corresponding post-logout redirect URIs and enable authorization code flow with PKCE S256. For local CloudGuard sign-in, set `CLOUDGUARD_AUTH_REQUIRED=true` with the verifier and public Keycloak settings. GRC requests `/api/auth/onboard` through its existing controls-service proxy.

Validation and rollout

Automated coverage includes Google/registration redirects and PKCE state, signup UI actions, forged/wrong-audience token rejection, verified-email enforcement, existing-member preservation, recovery after identity-service failure, and PostgreSQL concurrency/isolation. The static sign-up page and its sign-in navigation were also checked in a browser using mock public configuration.

To validate the live deployment, create a new Google user and a new email user, verify their emails, and confirm they each reach a separate organization as administrator. Repeat login and verify that neither creates a duplicate organization. Sign in as an existing viewer and

confirm its organization and role remain unchanged. Confirm that one organization's token cannot read the other's data.

Live Google/Keycloak configuration was not changed during implementation because deployment OAuth credentials were not supplied. The local configuration script and tests are ready; the environment settings above are required to activate the feature.

References: Keycloak JavaScript adapter and Keycloak server administration.